

Computer Simulation

Lesson 12: Solver and ...

Takumi Miyoshi, Taku Yamazaki

Special thanks to: Dr. Pham Thi Huong

1

Lesson 12

Summary for the course

- Variables
 - Persistent variables
- Functions
 - Anonymous functions
 - Function handles
- Toolbox in MATLAB
 - Optimization toolbox

2

Persistent Variables

- A local variable will be cleared when the function stops executing
→ Memory is released when the function ends
- For persistent variables, the variable still exists and retains its former value when the function ends

3

Example

```
function func1
% This function increments
a variable count

count = 0;
count = count + 1;
fprintf('The value of count
is %d\n',count)
```

```
function func2
% This function increments
a persistent variable count
persistent count
if isempty(count)
    count = 0;
end
count = count + 1;
fprintf('The value of count
is %d\n',count)
```

4

Practice 1 (posnum(num))

- Write a function to ask a user to enter a positive number, error check until the number is positive
- Display error as following:
 - The first time user enters wrong: Error1
 - The second time user enter wrong: Error2
 - The third time user enter wrong: Error3
 - And so on

5

Answer

```
function num = posnum
% Prompt user and error-check until the user enters a positive number
num = input('Enter a positive number: ');
while num < 0
    error(num)
    num = input('Enter a positive number: ');
end

function error(num)
persistent count
if isempty(count) %isempty determine whether array is empty
    count = 0;
end
count = count + 1;
fprintf('Error #%%d ...following instructions¥n', count);
```

6

Anonymous Functions

Syntax: **function_name** =@(arguments) **function_body**

Example: `cir_area=@(radius) pi*radius.^2;`

→ Calling: `cir_area(2)`

- One-line function
- Does not have to be saved in an M-file
- Can be created in the Command Window or in any script
- Advantage:
 - Simple
 - Reduce the number of M-file in a program
- To display function definition: `function_name`
- There is no arguments → still have to keep ()

Example: `function_name = @() function_body`

→ Calling: `function_name()`

7

Function Handles

`%plot x,y with y is a function like sin, cos`

```
function fnfnexamp(funh)
```

```
% Example of a function handle. The handle of a  
function is passed and that function of x is  
plotted
```

```
x = 0:.25:2*pi;
```

```
y = funh(x);
```

```
plot(x,y,'r*')
```

```
% The @ operator will get the handle of a  
function and store it in a variable
```

Calling 1: `fnsin = @sin →fnfnexamp(fnsin)`

Calling 2: `fnfnexamp(@sin)`

8

Optimization Toolbox

- What is Optimization Toolbox
- Where to use
- How to use
- Example

9

Introduction

- Optimization Toolbox is provided by MATLAB, including set of functions for finding parameters that optimize (minimize or maximize) objectives while satisfying constraints.
- The toolbox allows users to define the optimization problem with functions and matrices
- The toolbox solvers allow finding
 - Finding optimal solutions to continuous and discrete problems
 - Performing tradeoff analyses

10

Where to use

- Optimization toolbox supports functions for optimizing (minimize or maximize) objects while satisfying constraints:
 - Minimize risk
 - Maximize profits
- Check MATLAB toolbox: ver

11

How to use

- Specify a problem:
 - Representing the problem, define the objective function needs to minimize or maximize
- Specify constraints
- Choose a solver
 - Choose the most appropriate solver and algorithm
- Set other options

12

How to use: Constraints

- Equality constraints
 - Example: $5x_1 + 2x_2 = 10$
- Inequality constraints
 - Example: $2x_1^2 + 3x_2^2 < 20$
- Bounds
 - Example: $0 \leq x_1, x_2 \leq 10$
- An optimization problem can subject to one or many kinds of constraints

13

How to use: Solvers

Constraint Type	Objective Type				
	Linear	Quadratic	Least Squares	Smooth Nonlinear	Nonsmooth
None	n/a ($f = \text{const}$, or $\min = -\infty$)	quadprog , Information	mldivide , lsqcurvefit , lsqnonlin , Information	fminsearch , fminunc , Information	fminsearch , *
Bound	linprog , Information	quadprog , Information	lsqcurvefit , lsqnonlin , lsqnonneg , Information	fminbnd , fmincon , fseminf , Information	fminbnd , *
Linear	linprog , Information	quadprog , Information	lsqlin , Information	fmincon , fseminf , Information	*
General Smooth	fmincon , Information	fmincon , Information	fmincon , Information	fmincon , fseminf , Information	*
Discrete, with Bound or Linear	intlinprog , Information	*	*	*	*

* means relevant solvers are found in [Global Optimization Toolbox](#) (Global Optimization Toolbox) functions (licensed separately from Optimization Toolbox™ solvers).

14

Example: linear programming problem

- Finds the minimum of a problem specified by $f(x)$ subject to:

$$A * x \leq b,$$

$$Aeq * x = beq,$$

$$lb \leq x \leq ub.$$

f , x , b , beq , lb , and ub are vectors, and A and Aeq are matrices

15

Solution

- The `linprog.m` is used to minimize the problems in form of Linear programming
- `help linprog`

16

Methods

- Method 1: Write a script to call the program `linprog.m`
- Method 2: MATLAB → APPS → Optimization

17

linprog: Input/output arguments (1/2)

- The general form of calling `linprog.m`
`[x,fval,exitflag,output,lambda]=linprog(f,A,b,Aeq,beq,lb,ub,x0,options)`

Input arguments:

A: Matrix of inequality constraint

b: right hand side of the inequality constraints

Aeq: Matrix of equality constraints

beq: right hand side of the equality constraints

lb,[]: $b \leq x$: lower bounds for x, no lower bound

ub,[]: $x \leq ub$: upper bounds for x, no upper bounds

x0: Start vector for the algorithm, if known, else []

options: options are set to determine what algorithm to use, etc.

18

linprog: Input/output arguments (2/2)

- The general form of calling linprog.m

`[x,fval,exitflag,output,lambda]=linprog(f,A,b,Aeq,beq,lb,ub,x0,options)`

Output arguments:

- x: optimal solution
- fval: optimal value of the objective function
- exitflag: tells whether the algorithm converged or not, exitflag > 0 means convergence
- output: a structure for number of iterations, algorithm used and PCG iterations (when LargeScale=on)
- lambda: a structure containing the Lagrange multipliers corresponding to the constraints.

19

Practice 2

- Find x that minimizes $f(x) = -2x_1 - 3x_2$ subject to
- $x_1 + 2x_2 \leq 8$
- $2x_1 + x_2 \leq 10$
- $x_2 \leq 3$
- $x_1, x_2 \geq 0$

20

Answer

- `f = [-2; -3];`
- `A = [1, 2; 2, 1; 0, 1]; % inequality constraints`
- `b = [8, 10, 3];`
- `lb = zeros(2,1); ub=[]; % bounds`
- `Aeq = []; beq = []; % no equality constraints`
- `options=[];`
- `x=linprog(f,A,b,Aeq,beq,lb,ub,x0,options)`
or `x=linprog(f,A,b,[],[],lb,[],[],[]) %see only result`
`[x,fval,exitflag] = linprog(f,A,b,Aeq,beq,lb,ub,x0,options)`
`[x,fval,flag,output,LagMult]=linprog(f,A,b,Aeq,beq,lb,ub,x0,options)`
`x = linprog(f,A,b,Aeq,beq,lb,ub,x0,optimset('Display','iter'))`

21

References

[1] Bo Gu, Computer Simulation course 2016

[2] Abebe Geletu, “Solving Optimization Problems using the Matlab Optimization Toolbox - a Tutorial”, Dec. 2007

[3]<https://www.mathworks.com/help/optim/index.html>

[4] Stormy Attaway, “MATLAB: A practical approach”, 2009

22

Final Assignment

- Create your own simulation:
 - Free topic: any kind of simulations
- Team Presentation on **July 24**:
 - 10 minutes for presentation
 - 5 minutes for Q&A
- Submission:
 - Submit your report (ppt + source code) to the Teams folder by **July 24**